Technical Report
Embedded System Project: Advanced PC FAN speed controller with overspeed alarm.
Author: Martins Osemudiamen Okoh.
TU Dublin ID: A0004717

Aim: Design and implementation of a Advanced PC fan speed controller using STM32L432KC microcontroller with overspeed alarm.

Objective:

1. Schematic design of the PC fan speed controller.
2. Software implementation.
3. Hardware implementation.
4. Testing / Debugging.
5. Documentation.

Design:

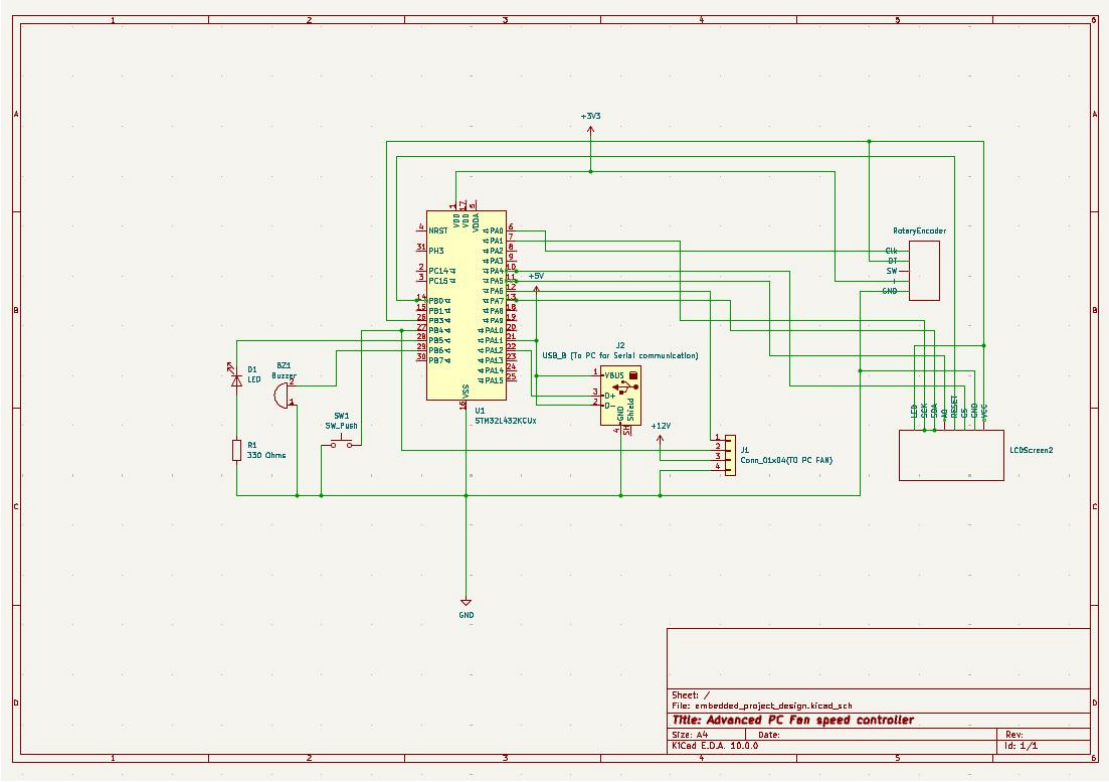

Figure 1: Ciricuit Design of PC fan controller using KICad 10.0.

In the Figure 1 above, the design comprises of a push button, an LED,330 Ohms resistor, Rotary Encoder, LCD Screen, Buzzer, PC fan 4 wire connector, a USB cable component for serial communication, voltage supplies of 3.3 and 12 voltages and a ground.

Source Code:

```
#include <stm32l432xx.h>
#include <stdint.h>
#include <math.h>
// added from serial file
#include <eeng1030_lib.h>
#include <stdio.h>
#include  <errno.h>
#include  <sys/unistd.h> // STDOUT_FILENO, STDERR_FILENO
#include "display.h"

void setup(void);
void delay(volatile uint32_t dly);
void enablePullUp(GPIO_TypeDef *Port, uint32_t BitNumber);
void pinMode(GPIO_TypeDef *Port, uint32_t BitNumber, uint32_t Mode);
void initADC(void);
int readADC(int chan);
void initTimer2(void);
void setTimer2Duty(int duty);
void initClocks(void);
void delay_ms(unsigned dly);
void initEncoder(void);
void initBuzzerTimer(void);
void initSerial(uint32_t baudrate);
void eputc(char c);
```

Figure 2: Declaration of functions and addition of necessary files and libraries.

Figure 2 above shows a hardware initialization and utility library for an STM32L432xx microcontroller (ARM Cortex-M4). It provides core functions for peripheral control, timing, analog reading, PWM generation, encoder interfacing, buzzer control, and serial communication. The code appears to be part of a larger embedded systems project (likely for a robotics or mechatronics course, given the eeng1030_lib.h reference).

```
// define needed constants
#define pulse_per_rev 2
#define BEEP_SAMPLES 4790
int16_t audioBuffer[BEEP_SAMPLES * 2];
volatile uint8_t playing = 0;// 0 = buzzer off, 1 = buzzer on (controlled by TIM6)

int vin, speed_hz, speed_rpm;
volatile unsigned pulse_count, milliseconds;
```

Figure 3: Declaration of global variables.

Figure 3 above shows the constants and variables for motor speed sensing and buzzer audio playback on an STM32 microcontroller. The constant pulse_per_rev is set to 2, meaning the motor encoder generates two pulses per revolution, while BEEP_SAMPLES defines the audio buffer size as 4790 samples. An int16_t array called audioBuffer allocates twice that many elements (9580 total) to store audio data for buzzer tones, and a volatile uint8_t flag named playing tracks whether the buzzer is active (1) or off (0), with updates controlled by Timer 6

interrupts. Integer variables vin, speed_hz, and speed_rpm hold motor voltage input and speed values in hertz and RPM respectively. Finally, two volatile unsigned variables—pulse_count for counting encoder pulses and milliseconds for system timing—are declared volatile because they are modified inside interrupt service routines but accessed in the main program loop.

```
int main()
{
    setup();
    SysTick->LOAD = 80000-1; // Systick clock = 80MHz. 80000000/80000=1000
    SysTick->CTRL = 7; // enable systick counter and its interrupts
    SysTick->VAL = 10; // start from a low number so we don't wait for ages for first interrupt

    initEncoder(); // TIM2 (input)
    init_display(); // SPI display
    initTimer16(); //TIM16 (PWM)
    initBuzzerTimer();      // TIM6 - buzzer tone generator

    delay_ms(2000);

    printf("MODER PA6: %lu\r\n",  (GPIOA->MODER >> 12) & 0x3);  // should be 2
    printf("AFR PA6:   %lu\r\n",  (GPIOA->AFR[0] >> 24) & 0xF); // should be 14
    printf("TIM16 CR1:  %lu\r\n", TIM16->CR1);   // should be 0x81
    printf("TIM16 CCER: %lu\r\n", TIM16->CCER);  // should be 0x1
    printf("TIM16 BDTR: %lu\r\n", TIM16->BDTR);  // should be 0x8000
    printf("TIM16 ARR:  %lu\r\n", TIM16->ARR);   // should be 3199
    printf("TIM16 CCR1: %lu\r\n", TIM16->CCR1);  // should be 1600
```

Figure 4: Main function of the software implementation part 1.

This main() function in Figure 4 above initializes the entire embedded system and then verifies the configuration by printing register values. It begins by calling setup() for basic hardware initialization, then configures the SysTick timer to generate interrupts every 1 millisecond (since an 80 MHz clock with a load value of 80,000 produces exactly 1000 Hz). It initializes the encoder on Timer 2, an SPI display, Timer 16 for PWM output, and Timer 6 for buzzer tone generation. After a 2-second delay to allow everything to stabilize, it prints the current register values for GPIO pin PA6 (checking its mode and alternate function), followed by several Timer 16 registers including CR1 (control), CCER (capture/compare enable), BDTR (break and dead-time), ARR (auto-reload value), and CCR1 (capture/compare value). These print statements serve as diagnostic checks to confirm that Timer 16 was configured correctly—for example, the ARR should be 3199 and CCR1 should be 1600, indicating a 50% duty cycle on the PWM output.

```
while(1)
{
    vin = TIM2->CNT;

    TIM16->CCR1 = (vin * 3199) / 159;

    //uncomment
    uint32_t pulses;

    __disable_irq();
    pulses = pulse_count;

    pulse_count = 0;

    //uncomment
    __enable_irq();

    // evaluate speed
    speed_hz = (pulses / pulse_per_rev);
    speed_rpm = (speed_hz * 450);
```

Figure 5: Main function of the software implementation part 2.

Figure 5 above shows continuous loop reads the motor's input voltage from Timer 2's counter register, scales it by a factor of 3199/159 (approximately 20.12) and writes the result to Timer 16's capture compare register to control a PWM output—likely for motor speed or LED brightness. It then safely copies the volatile pulse_count variable into a local pulses variable by disabling interrupts briefly to prevent corruption during the read, resets pulse_count to zero for the next measurement interval, and re-enables interrupts. Finally, it calculates the motor speed in hertz by dividing the accumulated pulses by pulse_per_rev (which is 2), then converts that to RPM by multiplying by 450—the conversion factor assumes the measurement period is 0.1333 seconds (since 60 seconds divided by 0.1333 equals 450). This loop continuously reads feedback, updates control outputs, and computes rotational speed in real time.

```
// LED: ON when speed is safe (<=3500), OFF when too fast
if (speed_rpm <= 3500)
{
    GPIOB->ODR |= (1 << 5);  // turn LED on – normal speed
    playing = 0;             // buzzer off
}
else
{
    GPIOB->ODR &= ~(1 << 5); // turn LED off – overspeed
    playing = 1;               // buzzer on via TIM6

}
```

Figure 6: Main function of the software implementation part 3.

This code implemented in Figure 6 shows a safety monitoring system that controls an LED and buzzer based on motor speed. If speed_rpm is 3500 RPM or less (safe zone), it turns the LED on by setting bit 5 of GPIOB's output data register, and ensures the buzzer is off by setting playing = 0. If the speed exceeds 3500 RPM (overspeed condition), it turns the LED off by clearing bit 5, and activates the buzzer by setting playing = 1, which enables Timer 6 to generate an audible alarm tone. This provides both visual feedback (LED on/off) and audible warning (buzzer) to alert the user when the motor is running too fast.

```
// print fan speed
printf("ENC: %d, CCR1: %lu, RPM: %d,buzz: %lu\r\n", vin, TIM16->CCR1, speed_rpm,(GPIOB->ODR >> 6) & 1);

// Clear text areas for display
fillRectangle(0,10,160,20,RGBToWord(0,0,0));
fillRectangle(0,30,160,20,RGBToWord(0,0,0));

// Display values
char buffer[50];

sprintf(buffer, "pos: %3d", vin);
printText(buffer, 5, 10, RGBToWord(255,255,0), 0);

sprintf(buffer, "RPM: %4d", speed_rpm);
printText(buffer, 5, 30, RGBToWord(0,255,0), 0);

delay_ms(100);
```

Figure 7: Main function of the software implementation part 4.

This code in Figure 7 handles real-time data output and user interface updates. It first prints a status line over serial containing the encoder value (vin), the current PWM capture compare register value from Timer 16, the calculated motor speed in RPM, and the buzzer state (extracted from bit 6 of GPIOB's output data register). It then updates an SPI display by clearing two text areas (at Y-positions 10 and 20, and 30 and 40) using black rectangles. The encoder position (vin) is formatted into a string labeled "pos" and displayed in yellow text at the top, while the RPM value is formatted and displayed in green text below. Finally, a 100 millisecond delay sets the loop's update rate to approximately 10 Hz, providing stable, readable updates without overwhelming the system.

```
void setup(void)
{
    initClocks();
    RCC->AHB2ENR |= (1 << 0) | (1 << 1); // turn on GPIOA and GPIOB
    pinMode(GPIOB,3,1); // PB3 digital output
    pinMode(GPIOB,4,0); // PB4 digital input (tachometer)
    pinMode(GPIOB, 5, 1); // PB5 output for LED
    pinMode(GPIOB, 6, 1); // PB6 as output for buzzer
    enablePullUp(GPIOB,4); // pull-up for tach input
    pinMode(GPIOA,3,2);  // alternative function mode

    GPIOA->AFR[0] &= ~(3 << (2*3));    // Clear out old alternative function  bits
    GPIOA->AFR[0] |= 1 << (4*3);    // Select alternative function 1

    // serial communication
    initSerial(9600); //115200

    // EXTI4 interrupt for tach pulse counting on PB4
    RCC->APB2ENR |= (1 << 0); // enable SYSCFG
    SYSCFG->EXTICR[1] &= ~(7 << 0); // clear perhaps previously set bits
    SYSCFG->EXTICR[1] |= (1 << 0);  // map EXTI2 interrupt to PB4
    EXTI->FTSR1 |= (1 << 4); // select falling edge trigger for PB4 input
    EXTI->IMR1 |= (1 << 4);  // enable PB4 interrupt
    NVIC->ISER[0] |= (1 << 10); // IRQ 10 maps to EXTI4
    __enable_irq();

}
```

Figure 8: Setup Function.

This setup() function shown in Figure 8 above performs all low-level hardware initialization for the system. It starts by calling initClocks() to configure system clocks, then enables GPIO ports A and B via the AHB2ENR register. It configures several GPIO pins: PB3 as a digital output, PB4 as a digital input for tachometer sensing (with a pull-up enabled), PB5 as an output for the status LED, and PB6 as an output for the buzzer. Pin PA3 is set to alternative function mode, and its alternate function is configured to AF1 (likely for timer input capture). Serial communication is initialized at 9600 baud for debugging output. The function then sets up an external interrupt on PB4 to count tachometer pulses: it enables the SYSCFG clock, configures the EXTI to map PB4 to interrupt line 4, selects falling edge triggering, enables the interrupt in the EXTI controller, and enables the EXTI4 interrupt in the NVIC (IRQ 10). Finally, it globally enables interrupts with __enable_irq(), allowing the tachometer pulse counting to begin.

```c
void delay(volatile uint32_t dly)
{
    while(dly--);
}

void enablePullUp(GPIO_TypeDef *Port, uint32_t BitNumber)
{
    Port->PUPDR = Port->PUPDR &~(3u << BitNumber*2); // clear pull-up resistor bits
    Port->PUPDR = Port->PUPDR | (1u << BitNumber*2); // set pull-up bit
}
void pinMode(GPIO_TypeDef *Port, uint32_t BitNumber, uint32_t Mode)
{
    /*
        Modes : 00 = input
                01 = output
                10 = special function
                11 = analog mode
    */
    uint32_t mode_value = Port->MODER;
    Mode = Mode << (2 * BitNumber);
    mode_value = mode_value & ~(3u << (BitNumber * 2));
    mode_value = mode_value | Mode;
    Port->MODER = mode_value;
}
```

Figure 9: delay , Pull Up resistor and Pinmode.

These three functions shown in Figure 9 above provide basic GPIO control and timing utilities.

`delay(volatile uint32_t dly)` creates a simple blocking delay by decrementing the input value in a tight while loop until it reaches zero. The parameter is declared `volatile` to prevent the compiler from optimizing away the loop.

`enablePullUp(GPIO_TypeDef *Port, uint32_t BitNumber)` activates the internal pull-up resistor on a specified GPIO pin. It clears the two bits controlling the pull configuration for that pin by ANDing with a mask, then sets the lower bit (binary 01) to select pull-up mode.

`pinMode(GPIO_TypeDef *Port, uint32_t BitNumber, uint32_t Mode)` configures the mode of a GPIO pin by writing to the port's MODER register. The Mode parameter (0=input, 1=output, 2=alternate function, 3=analog) is shifted left by twice the bit number to align with the two bits that control that pin, then combined into the register after clearing the existing configuration bits for that pin.

```
// I/O List for Encoder:
// PA0 : TIM2_CH1 (Alternative function 1) : Clock pin on encoder
// PB3 : TIM2_CH2 (Alternative function 1) : DT pin on encoder (quadrature type input)
// PB4 : GPIOA - SW input from encoder (maybe)
void initEncoder()
{
    RCC->AHB2ENR |= (1 << 0) | (1 << 1); // make sure GPIOA,B are turned on
    pinMode(GPIOA,0,2);
    selectAlternateFunction(GPIOA,0,1);
    pinMode(GPIOB,3,2);
    selectAlternateFunction(GPIOB,3,1);
    //pinMode(GPIOB,4,0);
    RCC->APB1ENR1 |= (1 << 0); // enable Timer 2
    TIM2->CR1 = 0;
    TIM2->CNT = 0;
    TIM2->ARR = 159; // full range encoder count

    // Encoder mode 3 (TI1 + TI2)
    TIM2->SMCR = 3;

    // TI1 and TI2 as inputs
    TIM2->CCMR1 = 0;
    TIM2->CCMR1 |= (1 << 0);  // CC1S = input
    TIM2->CCMR1 |= (1 << 8);  // CC2S = input
    TIM2->CCER = 0;
    TIM2->CR1 |= 1;

}
```

Figure 10: Timer 2.

This initEncoder() function configures Timer 2 shown in Figure 10 above in encoder mode to read a quadrature rotary encoder. It enables GPIOA and GPIOB clocks, then configures PA0 as TIM2 channel 1 (clock pin) and PB3 as TIM2 channel 2 (DT pin) both in alternate function mode with AF1 selected. Timer 2 is enabled, its counter is reset to zero, and the auto-reload register ARR is set to 159, which limits the count range from 0 to 159. The SMCR register is set to mode 3, which configures the timer to count on both TI1 and TI2 edges (quadrature decoding). The CCMR1 register sets both capture compare channels 1 and 2 as inputs, and CCER is cleared to configure the inputs. Finally, the timer is started by setting bit 0 in CR1. As the encoder rotates, the timer count automatically increments or decrements based on direction, and the value can be read from TIM2->CNT.

```c
void initTimer16()
{
    RCC->AHB2ENR |= (1 << 0);          // GPIOA clock on — must be before AFR access
    RCC->APB2ENR |= (1 << 17);         // TIM16 clock on

    // Manually configure PA6 as AF14 — do NOT use selectAlternateFunction
    GPIOA->AFR[0] &= ~(0xF << 24);     // clear PA6 AF bits (bits 27:24)
    GPIOA->AFR[0] |=  (14  << 24);     // set AF14

    GPIOA->MODER  &= ~(3u << 12);      // clear PA6 mode bits
    GPIOA->MODER  |=  (2u << 12);      // set alternate function mode

    // CHECK IMMEDIATELY AFTER WRITING
    printf("MODER inside initTimer16: %lu\r\n", (GPIOA->MODER >> 12) & 0x3);

    TIM16->CR1   = 0;
    TIM16->PSC   = 0;
    TIM16->ARR   = 3199;               // 80MHz / 3200 = 25kHz
    TIM16->CCMR1 = (0b110 << 4) | (1 << 3) | (1 << 2);
    TIM16->CCER  = (1 << 0);           // CC1E
    TIM16->CCR1  = 1600;               // 50% duty
    TIM16->BDTR  = (1 << 15);          // MOE
    TIM16->EGR  |= (1 << 0);
    TIM16->CR1   = (1 << 7) | (1 << 0);

    // CHECK AGAIN AT END OF FUNCTION
    printf("MODER end of initTimer16: %lu\r\n", (GPIOA->MODER >> 12) & 0x3);
}
```

Figure 11: Timer 16.

This initTimer16() function shown in Figure 11 configures Timer 16 to generate a PWM signal on pin PA6. It enables the GPIOA clock and Timer 16 clock, then manually configures PA6 to use alternate function 14 (AF14), which is the timer output channel for TIM16. The pin mode is set to alternate function. Diagnostic printf statements verify the pin's mode register value immediately after configuration. The timer itself is then set up: prescaler is 0, the auto-reload register ARR is 3199, which with an 80 MHz clock produces a PWM frequency of 25 kHz. The CCMR1 register configures channel 1 for PWM mode 2 (110 binary) with preload enabled, and CCER enables the capture/compare output. CCR1 is set to 1600, giving a 50% duty cycle. The BDTR register's MOE (Main Output Enable) bit is set, which is required for advanced timers to enable PWM output. An update event generates reloads the settings, and CR1 enables auto-reload preload and starts the timer counter.

```c
void initBuzzerTimer(void)
{
    RCC->APB1ENR1 |= (1 << 4);   // enable TIM6 clock

    TIM6->CR1  = 0;
    TIM6->PSC  = 7999;           // 80MHz / 8000 = 10kHz
    TIM6->ARR  = 9;              // 10kHz / 10 = 1kHz toggle → 500Hz buzz tone
    TIM6->DIER = (1 << 0);       // enable update interrupt
    TIM6->EGR  = (1 << 0);       // generate update event
    TIM6->CR1  = (1 << 0);       // start timer

    NVIC->ISER[1] |= (1 << 22);  // TIM6 IRQ = 54, bit 22 of ISER[1]
}
```

Figure 12: Timer 6.

This initBuzzerTimer() function shown in Figure 12 above configures Timer 6 to generate an interrupt for producing an audible buzzer tone. It enables the TIM6 clock via the APB1ENR1 register, then sets the prescaler to 7999, which divides the 80 MHz system clock down to 10 kHz. The auto-reload register ARR is set to 9, meaning the timer counts from 0 to 9 and then resets, creating an update event at 1 kHz (since 10 kHz divided by 10 equals 1000 Hz). This triggers an interrupt twice per cycle, allowing the buzzer pin to toggle at 500 Hz, which produces an audible 500 Hz tone. The DIER register enables the update interrupt, an update event is generated to load the prescaler and ARR values, and the timer is started by setting bit 0 in CR1. Finally, the interrupt is enabled in the NVIC at IRQ number 54 (bit 22 in ISER[1]), allowing the buzzer control to be handled in the TIM6 interrupt service routine.

```
// ------------------------------------------------------------
// TIM6 ISR - toggles PB6 to generate buzzer tone
// ------------------------------------------------------------
void TIM6_DAC_IRQHandler(void)
{
    TIM6->SR &= ~(1 << 0);            // clear update interrupt flag

    if (playing)
    {
        GPIOB->ODR ^= (1 << 6);       // toggle PB6 → 500Hz tone on buzzer
    }
    else
    {
        GPIOB->ODR &= ~(1 << 6);      // ensure buzzer pin is LOW when off
    }
}
```

Figure 13: Timer 6 interrupt .

This is the interrupt service routine for Timer 6 shown in Figure 13 which generates the buzzer tone. When a timer update interrupt occurs, the handler first clears the interrupt flag by writing 0 to bit 0 of TIM6's status register (SR). It then checks the global playing flag: if playing is 1 (buzzer active), it toggles pin PB6 by XORing bit 6 of GPIOB's output data register, creating a square wave at 500 Hz and producing an audible tone. If playing is 0 (buzzer off), it ensures the buzzer pin is driven low by clearing bit 6, keeping the buzzer silent. This ISR runs at 1000 Hz (every 1 ms), toggling the pin on each interrupt to produce the 500 Hz tone only when the overspeed condition is active.

```
117  void delay(volatile uint32_t dly)
118  {
119      while(dly--);
120  }
```

Figure 14: Function of installing delay.

```
122  void enablePullUp(GPIO_TypeDef *Port, uint32_t BitNumber)
123  {
124      Port->PUPDR = Port->PUPDR &~(3u << BitNumber*2); // clear pull-up resistor bits
125      Port->PUPDR = Port->PUPDR | (1u << BitNumber*2); // set pull-up bit
126  }
```

Figure 15: Function to enable pull-up resistor.

```c
void pinMode(GPIO_TypeDef *Port, uint32_t BitNumber, uint32_t Mode)
{
    /*
        Modes : 00 = input
                01 = output
                10 = special function
                11 = analog mode
    */
    uint32_t mode_value = Port->MODER;
    Mode = Mode << (2 * BitNumber);
    mode_value = mode_value & ~(3u << (BitNumber * 2));
    mode_value = mode_value | Mode;
    Port->MODER = mode_value;
}
```

Figure 16: Function for PIN mode selection.

.

```c
void initClocks()
{
    // Initialize the clock system to a higher speed.
    // At boot time, the clock is derived from the MSI clock
    // which defaults to 4MHz.  Will set it to 80MHz
    // See chapter 6 of the reference manual (RM0393)
        RCC->CR &= ~(1 << 24); // Make sure PLL is off

        RCC->PLLCFGR = (1 << 25) + (1 << 24) + (1 << 22) + (1 << 21) + (1 << 17) + (80 << 8) + (1 << 0);
        RCC->CR |= (1 << 24); // Turn PLL on
        while( (RCC->CR & (1 << 25))== 0); // Wait for PLL to be ready
    // configure flash for 4 wait states (required at 80MHz)
        FLASH->ACR &= ~((1 << 2)+ (1 << 1) + (1 << 0));
        FLASH->ACR |= (1 << 2);
        RCC->CFGR |= (1 << 1)+(1 << 0); // Select PLL as system clock
}
```

Figure 17: Function for initializing clock

```c
// added from serial communication file
void initSerial(uint32_t baudrate)
{
    RCC->AHB2ENR |= (1 << 0); // make sure GPIOA is turned on
    pinMode(GPIOA,2,2); // alternate function mode for PA2
    selectAlternateFunction(GPIOA,2,7); // AF7 = USART2 TX
    pinMode(GPIOA,15,2);
    selectAlternateFunction(GPIOA,15,3);
    RCC->APB1ENR1 |= (1 << 17); // turn on USART2

    const uint32_t CLOCK_SPEED=80000000;
    uint32_t BaudRateDivisor;

    BaudRateDivisor = CLOCK_SPEED/baudrate;
    USART2->CR1 = 0;
    USART2->CR2 = 0;
    USART2->CR3 = (1 << 12); // disable over-run errors
    USART2->BRR = BaudRateDivisor;
    USART2->CR1 =  (1 << 3);  // enable the transmitter and receiver
    USART2->CR1 |=  (1 << 2);  // enable the transmitter and receiver
    USART2->CR1 |= (1 << 0);
}
```

Figure 18: Function of initializing serial communication and pin definition.

```
197  int _write(int file, char *data, int len)
198  {
199      if ((file != STDOUT_FILENO) && (file != STDERR_FILENO))
200      {
201          errno = EBADF;
202          return -1;
203      }
204      while(len--)
205      {
206          eputc(*data);
207          data++;
208      }
209      return 0;
210  }
```

Figure 19: Function for writing via serial communication.

```
211  void eputc(char c)
212  {
213      while( (USART2->ISR & (1 << 6))==0); // wait for ongoing transmission to finish
214      USART2->TDR=c;
215  }
```

Figure 20: Function for write via serial communication to terminal display.

```
217  void EXTI4_IRQHandler()
218  {
219      //PIOB->BSRR = (1 << 3); // set PB3 to turn on LED
220      //GPIOB->IDR |= (1 << 4);
221      EXTI->PR1 = (1 << 4);   // clear interrupt pending flag
222      pulse_count++; //counting the pulse
223
224  }
```

Figure 21: Function to handle interrupt to counting the pulse for the speed estimation of the PC fan.

```
225  void SysTick_Handler(void)
226  {
227      milliseconds++;
228  }
229  void delay_ms(unsigned dly){
230      unsigned end = milliseconds + dly;
231      while(milliseconds != end);
232  }
```

Figure 22: Function for milliseconds counting and delays.

Testing:

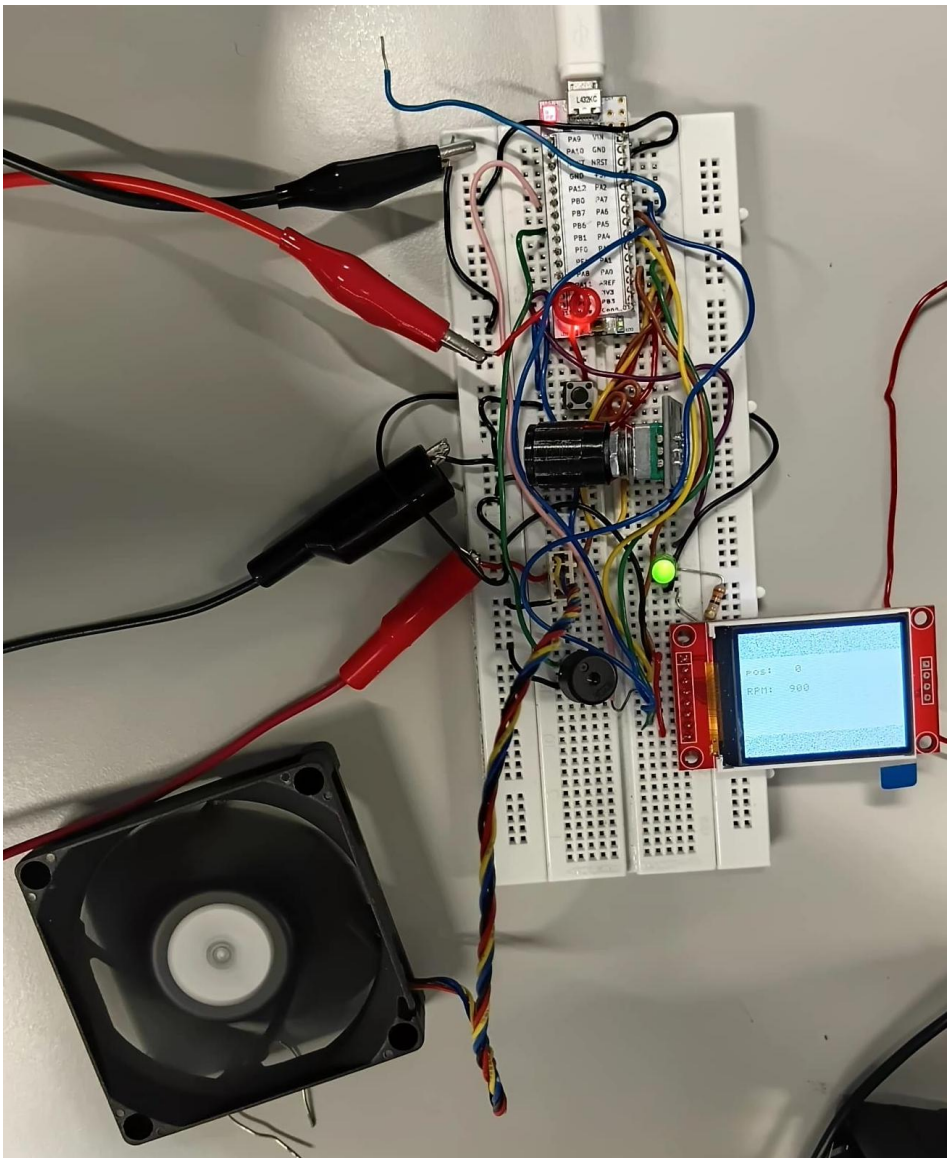

Figure 23: Hardware implementation of bread board.

The whole assemble is shown above in Figure 23 where the PC fan controller is being power,programmed and debugged.

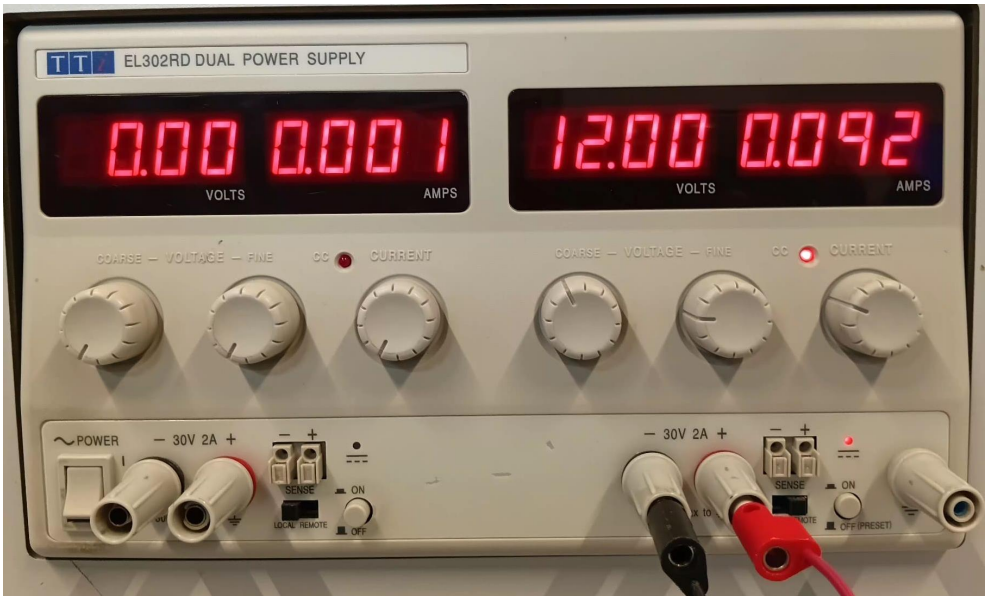

Figure 24: Power supply.

```
--- Quit: Ctrl+C | Menu: Ctrl+T | Help: Ctrl+T followed by Ctrl+H
MODER PA6: 2
AFR PA6:   14
TIM16 CR1:  129
TIM16 CCER: 1
TIM16 BDTR: 32768
TIM16 ARR:  3199
TIM16 CCR1: 1600
```

Figure 25: Output result 1 on terminal.

The printed diagnostic values shown in Figure 25 confirm that Timer 16 and its associated GPIO pin PA6 have been configured correctly for PWM output. PA6's mode register reads 2, indicating alternate function mode, and its alternate function register reads 14, which correctly maps to TIM16 channel 1. Timer 16's control register CR1 shows 129 (binary 10000001), meaning the counter is enabled with auto-reload preload activated. The capture/compare enable register CCER reads 1, confirming the output channel is enabled, while the break and dead-time register BDTR reads 32768 (binary 1000000000000000), indicating the main output enable bit is set—a critical requirement for PWM output on advanced timers. Finally, the auto-reload register ARR contains 3199, producing a 25 kHz PWM signal from the 80 MHz clock, and the capture/compare register CCR1 contains 1600, giving a precise 50% duty cycle. All values match the expected configuration, confirming TIM16 is fully operational and ready to generate PWM signals on pin PA6.

```
ENC: 0, CCR1: 0, RPM: 900,buzz: 0
ENC: 0, CCR1: 0, RPM: 900,buzz: 0
ENC: 0, CCR1: 0, RPM: 900,buzz: 0
ENC: 0, CCR1: 0, RPM: 900,buzz: 0
ENC: 0, CCR1: 0, RPM: 900,buzz: 0
ENC: 0, CCR1: 0, RPM: 900,buzz: 0
ENC: 0, CCR1: 0, RPM: 900,buzz: 0
ENC: 0, CCR1: 0, RPM: 900,buzz: 0
ENC: 0, CCR1: 0, RPM: 900,buzz: 0
```

Figure 26: Output result 2 on terminal.

The result in Figure 26 shows the speed of max of 900 revolution per minute when the potentiometer is at minimum level.

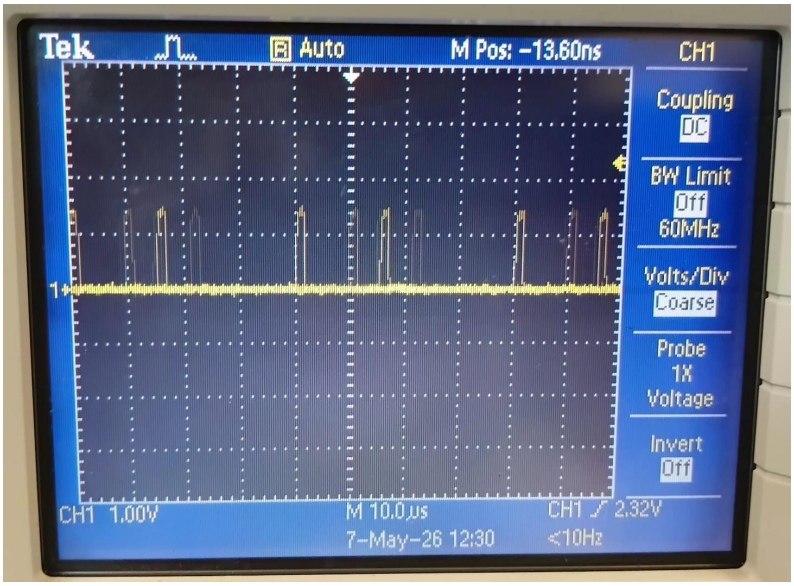

Figure 27: Fan output speed signal 1 viewed on oscilloscope.

The Figure 27 shows the duty cycle at 900 RPM which is almost 0% duty cycle.

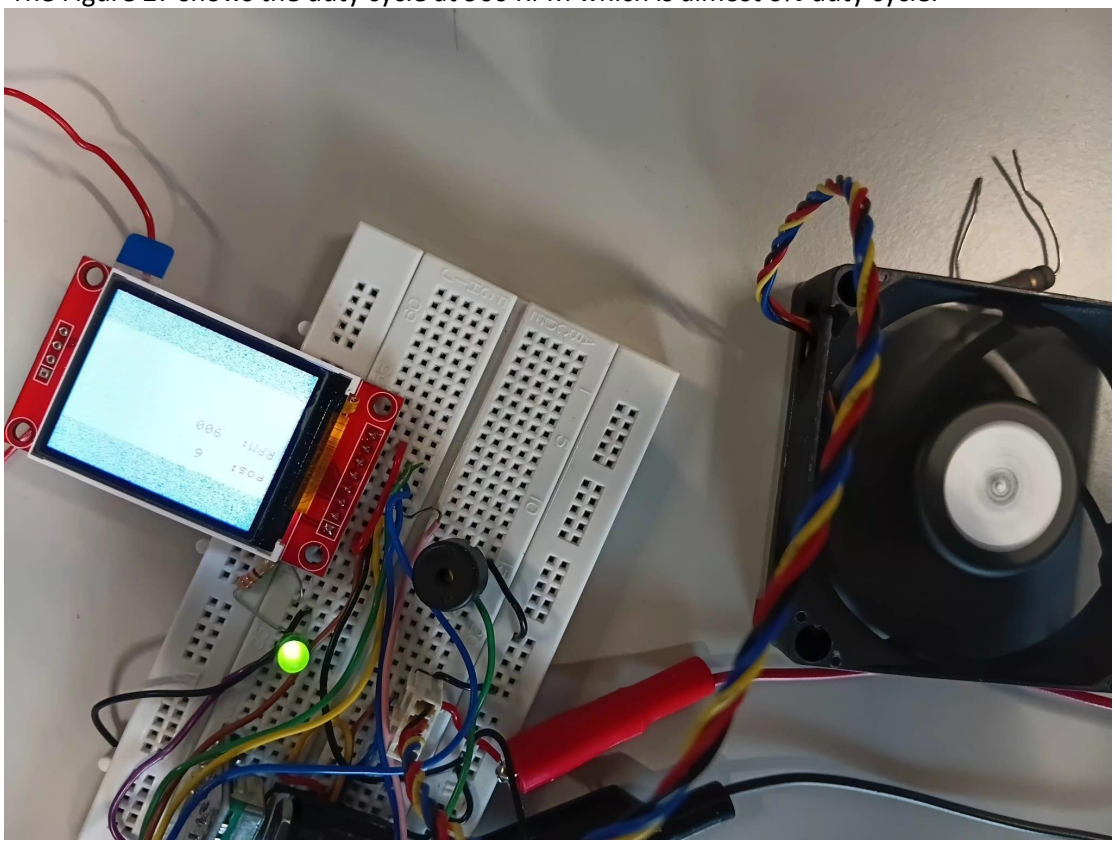

Figure 28: PC fan Assemble with LED ON and Screen .

The result in Figure 28 shows the output speed signal from the PC fan with LED ON, Fan ON and Screen indicating rotary position: 6 and speed 900 RPM.

```
ENC: 149, CCR1: 2997, RPM: 4050,buzz: 0
ENC: 149, CCR1: 2997, RPM: 3600,buzz: 1
ENC: 149, CCR1: 2997, RPM: 4050,buzz: 0
ENC: 149, CCR1: 2997, RPM: 4050,buzz: 1
ENC: 149, CCR1: 2997, RPM: 3600,buzz: 0
ENC: 149, CCR1: 2997, RPM: 3600,buzz: 1
ENC: 149, CCR1: 2997, RPM: 4050,buzz: 0
ENC: 149, CCR1: 2997, RPM: 3600,buzz: 1
ENC: 149, CCR1: 2997, RPM: 4050,buzz: 0
ENC: 149, CCR1: 2997, RPM: 4050,buzz: 1
ENC: 149, CCR1: 2997, RPM: 4050,buzz: 0
```

Figure 29: Output result 2 on terminal.

The result in Figure 29 shows rotary postion:149 with speed minimum of 3600 and buzzer toggling between 0 and 1.

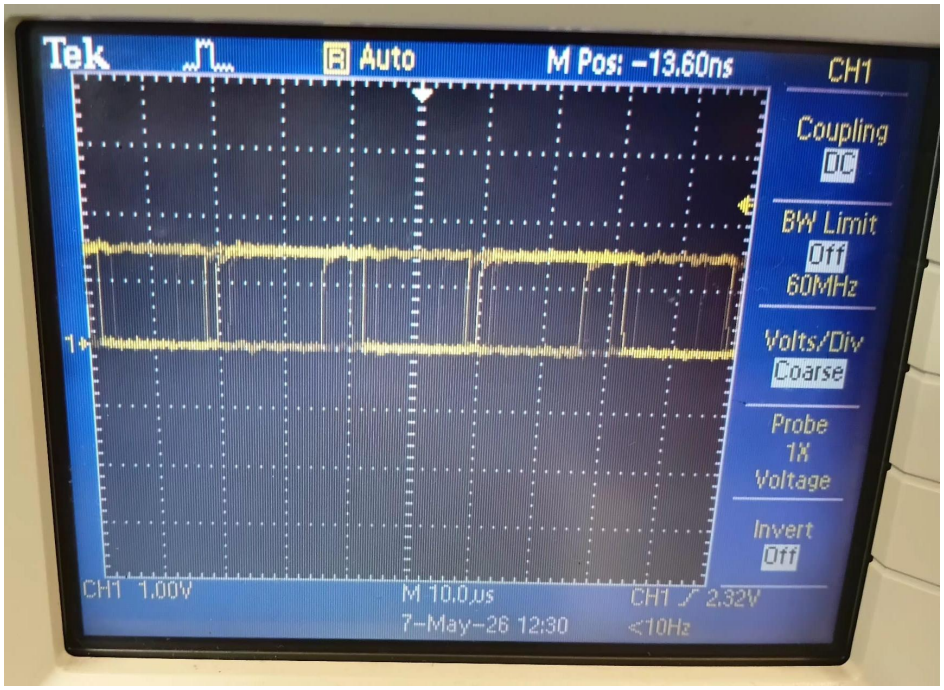

Figure 30: Fan output speed signal 2 viewed on oscilloscope.

The Figure 30 shows the duty cycle at minimum 3600 RPM which is almost 100% duty cycle.

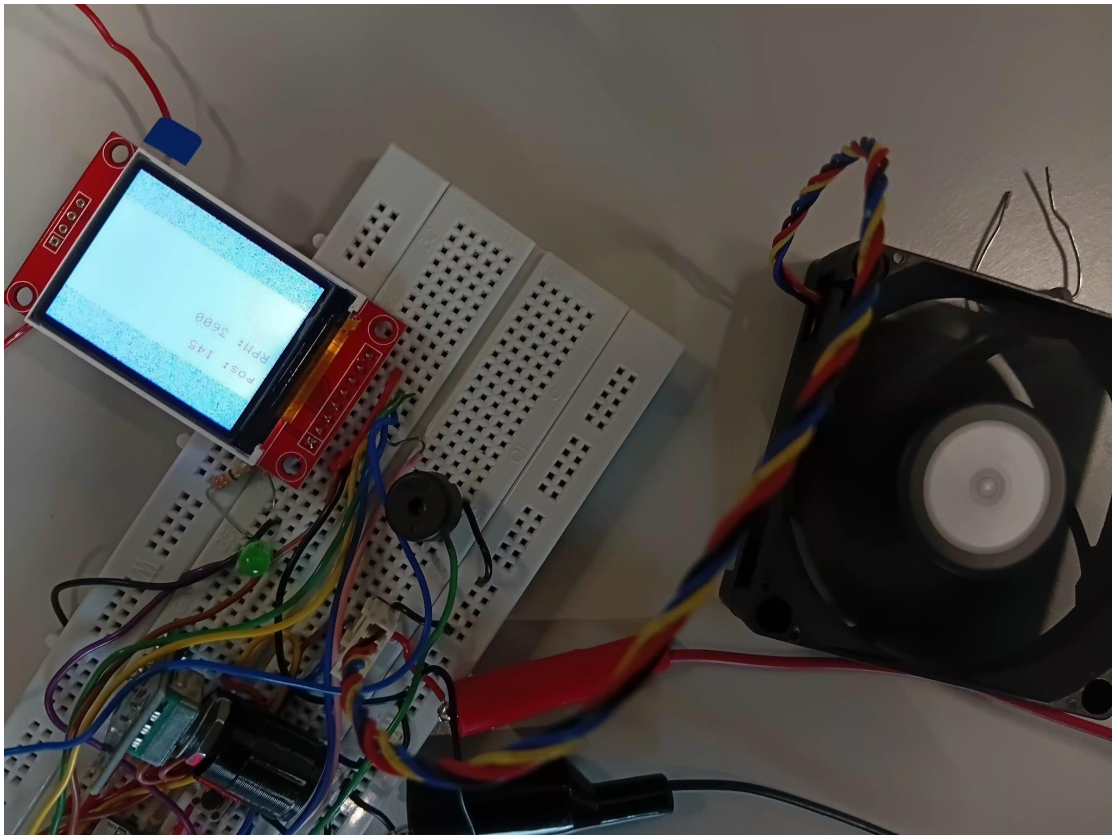

Figure 31: PC fan Assemble with LED OFF, Buzzer toggling and Screen .

The result in Figure 31 shows the output speed signal from the PC fan with LED OFF, Fan ON and at maximum and Screen indicating rotary position: 145 and speed 3600 RPM.

Conclusion:

The Advanced PC Fan Speed Controller using the STM32L432KC microcontroller was successfully designed, implemented, and tested. The project achieved all objectives, including schematic design, software and hardware implementation, testing, debugging, and documentation. The system effectively controlled fan speed through PWM generation, monitored RPM using tachometer feedback, and provided real-time user interaction through the LCD display and serial communication. Safety features such as LED indication and buzzer alerts operated correctly during over speed conditions. Experimental results confirmed accurate PWM control, reliable speed measurement, and stable system performance, demonstrating the effectiveness of the STM32-based embedded control system for PC fan applications.

References:

1) M. S. Waghare, D. R. Tutakne, A. N. Deshmukh, and M. M. Mardikar, “PWM controlled high power factor single phase fan regulator,” in Proc. 3rd Int. Conf. Electronics, Communication and Aerospace Technology (ICECA), 2019, pp. 59–64, doi: 10.1109/ICECA.2019.8822156.